COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	October 21, 2026	**DATE SUBMITTED:**	October 21, 2026
NAME:	Bungag, Marc Jhowell Limpio, Ramelissa Melith	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Bubble Sort	O(n)	$O(n^2)$	O(n) in the best case (sorted input: n−1 comparisons) but $O(n^2)$ in the worst case (reverse sorted: n(n−1)/2 comparisons).
Merge Sort	O (n log n)	O (n log n)	Merge Sort is O(n log n) in both cases, at the cost of O(n) extra space.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Bubble Sort Comparison / Time (ms)	Merge Sort Comparison / Time (ms)
Best, n = 100	99 / 0.0008ms	356 / 0.0136ms
Best, n = 1,000	999 / 0.0062ms	5,044 / 0.1675ms
Best, n = 10,000	9,999 / 0.0602ms	69,008 / 2.1302ms
Worst, n = 100	4,950 / 0.0864ms	316 / 0.0135ms
Worst, n = 1,000	499,500 / 8.6176ms	4,932 / 0.1555ms
Worst, n = 10,000	49,995,000 / 907.8230ms	64,608 / 2.0356ms

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

```cpp
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;
using Clock = chrono::high_resolution_clock;

long long cmpCount = 0;

// Bubble sort with early exit (best case O(n) on sorted input)
void bubbleSort(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            cmpCount++;
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

void merge(vector<int>& a, vector<int>& tmp, int l, int m, int r) {
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        cmpCount++;
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else tmp[k++] = a[j++];
    }
    while (i <= m) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    for (int x = l; x <= r; x++) a[x] = tmp[x];
}

void mergeRec(vector<int>& a, vector<int>& tmp, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeRec(a, tmp, l, m);
    mergeRec(a, tmp, m + 1, r);
    merge(a, tmp, l, m, r);
}

void mergeSort(vector<int>& a) {
    vector<int> tmp(a.size());
    mergeRec(a, tmp, 0, (int)a.size() - 1);
}

template <class F>
void run(const char* name, F sorter, vector<int> data, const char* caseName, int n) {
    cmpCount = 0;
    auto t0 = Clock::now();
    sorter(data);
    auto t1 = Clock::now();
    double ms = chrono::duration<double, milli>(t1 - t0).count();
    bool ok = is_sorted(data.begin(), data.end());
    cout << left << setw(12) << name << setw(8) << caseName << right << setw(8) << n
         << setw(14) << cmpCount << setw(14) << fixed << setprecision(4) << ms
         << (ok ? "  sorted" : "  FAILED") << "\n";
}

int main() {
    cout << left << setw(12) << "Algorithm" << setw(8) << "Case" << right << setw(8) << "n"
         << setw(14) << "Comparisons" << setw(14) << "Time (ms)" << "\n";
    for (int n : {100, 1000, 10000}) {
        vector<int> best(n);
        iota(best.begin(), best.end(), 1);            // Already sorted
        vector<int> worst(best.rbegin(), best.rend()); // Reverse sorted
        run("Bubble", bubbleSort, best, "Best", n);
        run("Bubble", bubbleSort, worst, "Worst", n);
        run("Merge", mergeSort, best, "Best", n);
        run("Merge", mergeSort, worst, "Worst", n);
        cout << "\n";
    }
}
```

Table 14-3. Code of the Experimental Analysis

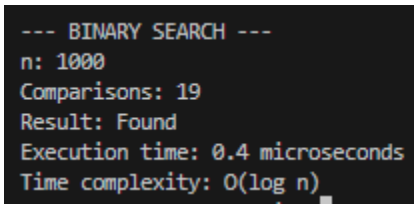
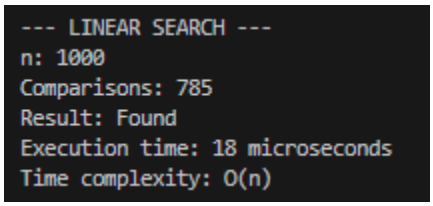
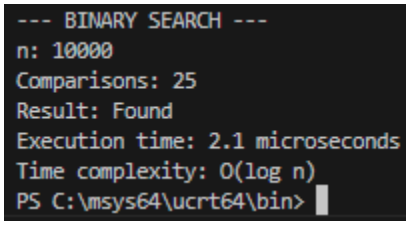
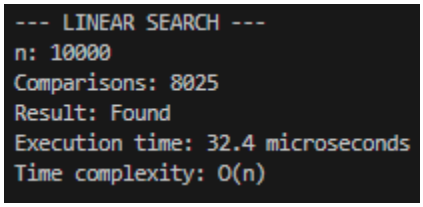
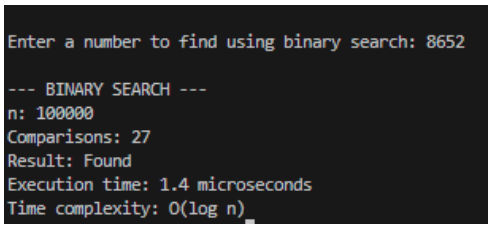
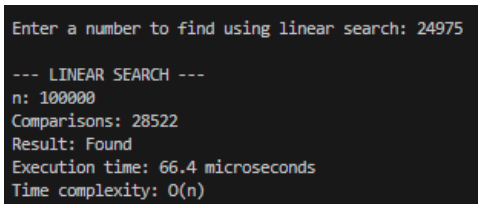
Algorithm	Case	n	Comparisons	Time (ms)	
Bubble	Best	100	99	0.0008	sorted
Bubble	Worst	100	4950	0.0864	sorted
Merge	Best	100	356	0.0136	sorted
Merge	Worst	100	316	0.0135	sorted
Bubble	Best	1000	999	0.0062	sorted
Bubble	Worst	1000	499500	8.6176	sorted
Merge	Best	1000	5044	0.1675	sorted
Merge	Worst	1000	4932	0.1555	sorted
Bubble	Best	10000	9999	0.0602	sorted
Bubble	Worst	10000	49995000	907.8230	sorted
Merge	Best	10000	69008	2.1302	sorted
Merge	Worst	10000	64608	2.0356	sorted

Table 14-4. Output of the Experimental Analysis

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode Analysis	
Linear Search	LinearSearch(A, n, key) for i = 0 to n - 1 if A[i] == key return i return -1
Binary Search	BinarySearch(A, n, key) low = 0 high = n − 1 while low ≤ high mid = (low + high) / 2 if A[mid] == key return mid else if A[mid] < key low = mid + 1 else high = mid − 1 return −1

Space and Time Complexity		
	Linear Search	**Binary Search**
Best case time	O(1)	O(1)
Average case time	O(n)	O(log n)
Worst case time	O(n)	O(log n)

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000		
10,000		
100,000		

Conclusion

Bungag
In conclusion, the experiments show that Bubble Sort is suitable for small or already sorted datasets, while Merge Sort is more efficient for larger datasets. For searching algorithms, Linear Search has a time complexity of $O(n)$, while Binary Search has $O(\log n)$ and is faster for large datasets when the data is sorted. Overall, choosing the right algorithm improves performance and reduces execution time.

Limpio
Growth rate matters more than speed at small sizes. Bubble Sort is O(n²) in the worst case, while Merge Sort stays O(n log n), and at n = 10,000 Bubble Sort was about 450× slower. Merge Sort's tradeoff is O(n) extra space. I did well on this activity, and I need more practice applying the same analysis to searching algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.